

Manual Maestro: Git, GitHub y Colaboración

Smartphone Use Prediction UTEC | Guía integral del proyecto, resolución de casos y convenciones.

1. Conceptos Clave (Resumen)

- **Git:** Programa en tu computadora que guarda el historial de cambios.
- **GitHub:** Nube donde se junta el código de todos.
- **Ramas (Branches):** Caminos paralelos. **Nunca se trabaja en `main`**. Cada tarea tiene su propia rama.

2. Día 1: Clonar el Repositorio (Una sola vez)

Para traer el proyecto de la nube a tu PC.

Bash

```
# 1. Descargar el repositorio (solo se hace la primera vez)
git clone https://github.com/frankrubio/Smarthphone-use-prediction-UTEC.git

# 2. Entrar a la carpeta (Si no entras, Git no sabrá dónde estás)
cd Smarthphone-use-prediction-UTEC
```

3. El Flujo de Trabajo Diario (Paso a Paso)

A. Iniciar una NUEVA tarea

Asegúrate de estar en `main` y tener la última versión antes de crear tu rama.

Bash

```
# 1. Traer lo último del equipo
git checkout main
git pull origin main

# 2. Crear y moverte a la nueva rama (ej. feature-limpieza)
git checkout -b feature-nombre-de-tu-tarea
```

B. Continuar trabajando en una rama EXISTENTE (Al día siguiente)

Si retomaste el trabajo de días anteriores o cambiaste de computadora, **siempre debes sincronizar tu rama antes de programar** para evitar conflictos.

Bash

```
# 1. Asegurarte de estar en tu rama de trabajo
git checkout feature-nombre-de-tu-tarea

# 2. DESCARGAR actualizaciones hechas en esta misma rama antes de tocar código
git pull origin feature-nombre-de-tu-tarea
```

💡 **Tras esto, puedes programar tranquilo**

Cierra Git Bash. Abre RStudio/VS Code. Apaga tu PC. Tus archivos están a salvo en tu disco.

C. Guardar y Subir tus avances

Haz esto cuando termines una funcionalidad lógica que valga la pena guardar.

Bash

```
# 1. Empaquetar archivos modificados
git add .

# 2. Registrar en el historial local con mensaje descriptivo
git commit -m "Agregar script de graficos EDA"

# 3. Subir a GitHub (Agrega '-u origin' SOLO la primera vez que subes la rama)
git push origin feature-nombre-de-tu-tarea
```

D. Revisión (Pull Request) y Merge

1. Entra a GitHub y presiona **"Compare & pull request"**.
2. Un compañero revisa. Luego presiona **"Merge pull request"**.
3. Presiona **"Delete branch"** en GitHub.

E. Sincronización y Limpieza Local

Bash

```
# 1. Vuelve a main y actualiza
git checkout main
git pull origin main

# 2. Borra tu rama local usada
git branch -d feature-nombre-de-tu-tarea

# 3. Limpia ramas borradas en GitHub de tu registro local
git fetch --prune
```

4. Casos Especiales y Errores (Solucionario)

Caso A: Modifiqué código pero olvidé hacer `pull` al iniciar el día

No hagas pull de golpe para no malograr tu código.

Bash

```
# 1. Guarda tus cambios en una caja temporal
git stash
# 2. Actualiza tu rama con seguridad
git pull origin tu-rama
# 3. Recupera tu código sobre la versión actualizada
git stash pop
```

Caso B: Trabajé código estando en `main` por error

Mientras no hayas hecho `git commit`, tus cambios están "flotando".

Bash

```
# Crea la rama correcta y muévete. Git trasladará tus cambios automáticamente.
git checkout -b tu-nueva-rama
```

Caso C: Continuar una rama activa de un compañero

Si un compañero subió una rama y quieres partir de allí (no de main):

Bash

```
# Mapea las nuevas ramas, luego crea la tuya apuntando a la del compañero
git fetch origin
git checkout -b mi-rama origin/rama-del-companero
```

Caso D: Error al subir (GH007: Private email address)

GitHub bloquea el push si tu cuenta oculta tu email. Solución:

Bash

```
# 1. Configura el correo noreply de GitHub en Git
git config user.email "ID+USER@users.noreply.github.com"
# 2. Reescribe el commit bloqueado y súbelo
git commit --amend --reset-author --no-edit
git push -u origin tu-rama --force
```

Caso E: Error "Updates were rejected" (Historiales no relacionados)

Ocurre al inicio si creaste el `README.md` en GitHub y difiere de tu PC.

Bash

```
# Si tienes los mismos archivos, sobrescribe GitHub con fuerza:
git push -u origin main --force
# O fusiona historiales (si ambos tienen cambios útiles):
git pull origin main --allow-unrelated-histories
```

Caso F: Advertencia LF will be replaced by CRLF

Es un aviso de que Git ajusta los saltos de línea de Linux a Windows. **No es un error.** Ignóralo o ejecuta: `git config --global core.autocrlf true`.

5. Resolviendo Conflictos de Fusión (Merge Conflicts)

Para evitar que tu rama falle al integrarse al final, absorbe los cambios de `main` periódicamente:

Bash

```
# Estando en tu rama, trae el código de main hacia ti
git merge main
```

⚠ ¿Qué hacer si hay conflicto?

Si dos personas editaron la misma línea, Git pausa todo. 1. Abre el archivo en VS Code / RStudio. 2. Verás `<<<<<< HEAD`, `=====`, y `>>>>>>`. 3. Borra esos marcadores y deja el código final limpio (como debe quedar). 4. Ejecuta: `git add .` → `git commit -m "Conflicto resuelto"`.

6. Estrategias en Documentos Quarto (.qmd)

- **Dividan por Secciones:** Repártanse partes (Intro, EDA, Modelos). Así no tocarán las mismas líneas.
- **Modularidad:** Creen archivos independientes y únanlos en un maestro con `{{< include parte1.qmd >}}`.
- **Commits Rápidos:** Suban avances pequeños a menudo; evitarán conflictos masivos inmanejables.

7. Entornos Virtuales (.venv y renv)

Para garantizar que todos usen las mismas librerías y versiones.

Python (venv)

Bash (En la raíz del proyecto)

```
# Crear, activar y exportar (Vía Terminal)
python -m venv .venv
source .venv/Scripts/activate
pip freeze > requirements.txt
```

En IDE: `Ctrl + Shift + P` → Python: Create Environment → Venv (carpeta `.venv`).

R (renv)

R Console

```
install.packages("renv")
renv::init()
```

🚫 Archivos Prohibidos (.gitignore)

Las carpetas pesadas `.venv/`, `renv/library/` y `__pycache__` **JAMÁS se suben**. Pónganlas en `.gitignore`. Solo suban `requirements.txt` y `renv.lock`.

8. Convenciones Obligatorias

Elemento	Regla	Ejemplo Correcto	Ejemplo Incorrecto
Archivos	<code>snake_case</code> , minúsculas, sin tildes ni espacios.	<code>01_limpieza.R</code>	<code>script final.R</code>
Commits	Imperativo y descriptivo.	<code>git commit -m "Corregir gráfica"</code>	<code>git commit -m "asdasd"</code>
Rutas	Estrictamente relativas al proyecto.	<code>read_csv("data/raw.csv")</code>	<code>read_csv("C:/Users/...")</code>
Seguridad	No subir contraseñas ni APIs. Usar <code>.env</code> .	<code>api_key = os.getenv("API")</code>	<code>api_key = "12345"</code>